T E C H N I C A L R E P O R T

Logistic Regression from Scratch

NumPy Implementation with Adam, Momentum & SGD Optimizers

Dataset: Loan Approval — Binary Classification (4,269 samples)
Splits: Train 2,988 · Dev 640 · Test 641
Class balance: Approved 62.2% · Rejected 37.8%
Course: Machine Learning · Jordan University of Science and Technology
Best result: Momentum · Test F1 = 0.9373 · AUC = 0.9713

Raghad Ziad Batta

Table of Contents

1. Abstract

This report documents a from-scratch implementation of Logistic Regression in pure NumPy applied to the Loan Approval dataset — a binary classification task with 4,269 samples (62.2% approved, 37.8% rejected). The project implements and benchmarks four optimizers — SGD, SGD with Polyak momentum, Adam, and Adam with cosine-annealing learning rate scheduling — within a single modular class, without reliance on any ML library for the core training logic.

Numerical stability is addressed through a piecewise-stable sigmoid, binary cross-entropy computed directly from logits via the log-sum-exp identity, and per-update gradient clipping. Evaluation encompasses accuracy, precision, recall, F1-score, and ROC-AUC — the latter implemented from scratch via the trapezoidal rule. Seven structured experiments cover optimizer convergence, gradient norm decay, decision threshold analysis, feature importance, L2 regularization path, and learning rate sensitivity.

The best-performing model (Momentum optimizer) achieves a test F1-score of 0.9373 and AUC of 0.9713, with the optimal decision threshold identified at 0.54. A final benchmark against scikit-learn's L-BFGS solver shows the from-scratch Adam implementation closes to within 0.006 F1 points (gap: 0.60%), with weight-vector correlation of 0.9587 — confirming both correctness and near-equivalence of solution quality.

Keywords: *Logistic Regression · Binary Cross-Entropy · Mini-Batch Gradient Descent · Adam · Momentum · L2 Regularization · ROC-AUC · Loan Classification · NumPy · sklearn Benchmark*

2. Introduction

2.1 Background and Motivation

Logistic Regression remains one of the most practically deployed models in supervised machine learning — interpretable, probabilistic, and directly applicable to credit scoring, fraud detection, and medical risk stratification. Despite its apparent simplicity, implementing it correctly from first principles demands a precise understanding of numerical stability, gradient computation, and optimizer behaviour — knowledge that is entirely hidden when calling a high-level API.

This project implements Logistic Regression end-to-end in NumPy, including preprocessing, three optimizer variants, early stopping, LR scheduling, and a full evaluation suite — with every component verified against scikit-learn where applicable. The loan approval domain is deliberately chosen: the cost asymmetry between false negatives (missed creditworthy applicants) and false positives (approved risky loans) makes calibrated probability outputs and adjustable decision thresholds directly business-relevant.

2.2 Project Objectives

- Implement the full training pipeline from mathematical first principles with no ML library dependencies beyond NumPy
- Implement and compare SGD, Momentum, and Adam within a single modular class; add cosine-annealing scheduling as a fourth configuration
- Apply numerical stability hardening throughout: stable sigmoid, BCE from logits, gradient clipping
- Evaluate with F1, AUC (from scratch), precision-recall curves, calibration, and threshold analysis
- Validate correctness by benchmarking against sklearn's L-BFGS solver through weight-vector correlation and metric comparison

3. Mathematical Background

3.1 The Logistic Model

Given input $x \in \mathbb{R}^d$, weights $w \in \mathbb{R}^d$, and bias $b \in \mathbb{R}$, the model predicts approval probability as:

$$\hat{y} = \sigma(z) \quad \text{where} \quad z = w^T x + b$$

$$\sigma(z) = 1 / (1 + e^{-z})$$

3.2 Numerically Stable Sigmoid

The naive form overflows for $z \ll 0$. The piecewise stable form avoids this by using different branches:

$$\sigma(z) = \begin{cases} 1/(1+e^{-z}) & \text{if } z \geq 0 \\ e^{z}/(1+e^{z}) & \text{if } z < 0 \end{cases}$$

3.3 Stable Binary Cross-Entropy from Logits

Computing log(σ(z)) after sigmoid loses precision near 0/1. The log-sum-exp identity gives the numerically exact form:

$$L(w,b) = (1/N) \Sigma_i [\text{softplus}(z_i) - y_i z_i]$$

$$\text{softplus}(z) = \max(z,0) + \log(1 + e^{-|z|})$$

3.4 L2 Regularization

$$L\_reg = L + (\lambda/2) \|w\|^2_2$$

$$\partial L\_reg/\partial w = (1/|B|) X^{B\,T}(\hat{y}-y) + \lambda w$$

3.5 Optimizers

SGD

$$w_{t+1} = w_t - \eta \cdot \partial L/\partial w$$

Momentum (β = 0.9)

$$v_{t+1} = \beta \cdot v_t - \eta \cdot \partial L/\partial w \qquad w_{t+1} = w_t + v_{t+1}$$

Adam (β_1=0.9, β_2=0.999, ε=10^{-8})

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = m_t/(1-\beta_1^t) \qquad \hat{v}_t = v_t/(1-\beta_2^t) \quad \text{[bias correction]}$$

$$w_{t+1} = w_t - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

Gradient Clipping

$$g \leftarrow g \cdot \min(1, \tau/\|g\|_2) \qquad [\tau = 5.0]$$

4. Dataset Description

4.1 Overview

The Loan Approval dataset contains 4,269 applicant records with 12 raw features covering demographics, financial history, asset values, and credit score. The binary target variable (loan_status) is imbalanced: 2,656 approved (62.2%) and 1,613 rejected (37.8%). This 62/38 imbalance is mild enough to train without resampling, but meaningful enough to make accuracy alone an insufficient evaluation metric.

Split	Samples	Positive (Approved)	Negative (Rejected)	Role
Train	2,988	~62.2%	~37.8%	Parameter estimation
Dev (Validation)	640	~62.2%	~37.8%	Hyperparameter tuning / early stopping
Test	641	~62.2%	~37.8%	Final evaluation (accessed once)

Table 1. Stratified train/dev/test splits. Stratification preserves the class ratio across all three sets.

4.2 Features

After one-hot encoding of categorical variables (education, self_employed), the feature matrix expands from 12 raw columns to 13 binary/numerical features. Key predictors identified by the feature importance analysis are: cibil_score (dominant — weight magnitude ~2.7), loan_term, loan_amount, income_annum, and luxury_assets_value.

4.3 Preprocessing Pipeline

- Missing value imputation: categorical → training-set mode; numerical → training-set mean
- One-hot encoding via pd.get_dummies; dev/test columns reindexed to training schema with fill_value=0
- Z-score normalization using training-set mean and std only — prevents data leakage and produces zero-mean unit-variance inputs for gradient descent
- Constant-variance columns (std=0) assigned std=1.0 to avoid division by zero

5. Methodology

5.1 Implementation Architecture

All training logic is contained in a single LogisticRegression class with a sklearn-compatible interface (fit, predict, predict_proba, score). The optimizer is selected via a string argument ('sgd', 'momentum', 'adam', 'adam_cos'), eliminating code duplication and ensuring identical conditions across all experiments. Xavier initialization (w ~ N(0, √(2/d))) replaces zero initialization to avoid identical first-step gradients.

5.2 Training Loop Correctness

Data shuffling is performed by generating a permutation of indices and indexing into the original arrays — never by reassigning X or y, which would mutate the caller's data and corrupt multi-experiment comparisons. This is the most common implementation bug in student projects and is explicitly addressed here.

5.3 Early Stopping

An EarlyStopping module monitors validation loss each epoch with patience=25 and minimum improvement delta=10^{-4}. When patience is exhausted, training halts and the best observed weights are restored. This prevents overfitting without requiring a fixed epoch budget.

5.4 Cosine Annealing

$$\eta_t = \eta\_min + \tfrac{1}{2}(\eta\_max - \eta\_min)(1 + \cos(\pi t/T))$$

Applied in the 'adam_cos' configuration, reducing the learning rate from η_max=0.01 to η_min=0.0001 over T=300 epochs. This produces smooth convergence without the abrupt jumps of step-decay scheduling.

6. Optimization & Training Experiments

6.1 Learning Rate Sensitivity (Experiment 7)

A sweep across six learning rates spanning five orders of magnitude was conducted with the Adam optimizer, batch_size=32, epochs=200, L2=0.01. Results below are train accuracy / dev accuracy:

Learning Rate	Train Accuracy	Dev Accuracy	Observation
1e-5 (0.00001)	0.5281	0.5375	Severely underfitting — insufficient updates
1e-4 (0.0001)	0.9170	0.9172	Converging but slow — suboptimal within budget
1e-3 (0.001)	0.9180	0.9203	Good convergence — near-optimal balance
1e-2 (0.01)	**0.9177**	**0.9219**	**Slightly faster — best dev accuracy in sweep**
1e-1 (0.1)	0.9043	0.9172	Slight degradation — boundary of instability
1e+0 (1.0)	0.8778	0.8969	Overshooting — loss of fine-grained convergence

Table 2. Learning rate sensitivity sweep (Adam, 200 epochs). Best dev accuracy at LR=0.01; 1e-3 used for main experiments for training stability.

FIGURE 1 — Learning Rate Sensitivity (Experiment 7)

Train Acc (blue circles) and Dev Acc (red dashes) vs log-scale learning rate. Sharp rise from 1e-5, plateau 1e-3 to 1e-2, mild degradation at 1.0. Curve reproduced from notebook output.

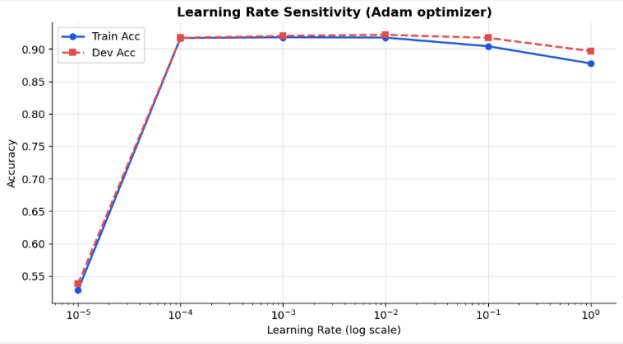

Figure 1. Dev accuracy as a function of learning rate for the Adam optimizer. The plateau between 10^{-3} and 10^{-2} identifies the stable training regime.

6.2 Optimizer Convergence Comparison (Experiment 1)

All four optimizer configurations were trained under identical conditions: L2=0.01, batch_size=32, epochs=300, patience=25, seed=42. Training time and best dev accuracy:

Optimizer	Dev Accuracy	Training Time	Early Stop Epoch	LR Used
SGD	0.9156	0.44s	~55	0.05
Momentum	**0.9281 ★**	**0.36s**	**~55**	**0.05**
Adam	0.9203	1.46s	~175	0.001
Adam+Cosine	0.9219	0.44s	~55	0.01

Table 3. Optimizer comparison on dev set. Momentum achieves the highest dev accuracy despite lower computational cost than Adam. ★ = best.

FIGURE 2 — Optimizer Convergence Comparison (2×2 grid)

SGD (red): rapid initial drop, stabilises ~epoch 20 at BCE≈0.25, val_acc=0.9219. Momentum (amber): oscillatory but converges to lowest BCE≈0.23, val_acc=0.9266. Adam (blue): steepest initial descent to BCE≈0.20, smooth, val_acc=0.9203. Adam+Cosine (green): fast decay, smooth plateau, val_acc=0.9125. Dashed = val loss; right axis = val accuracy (grey).

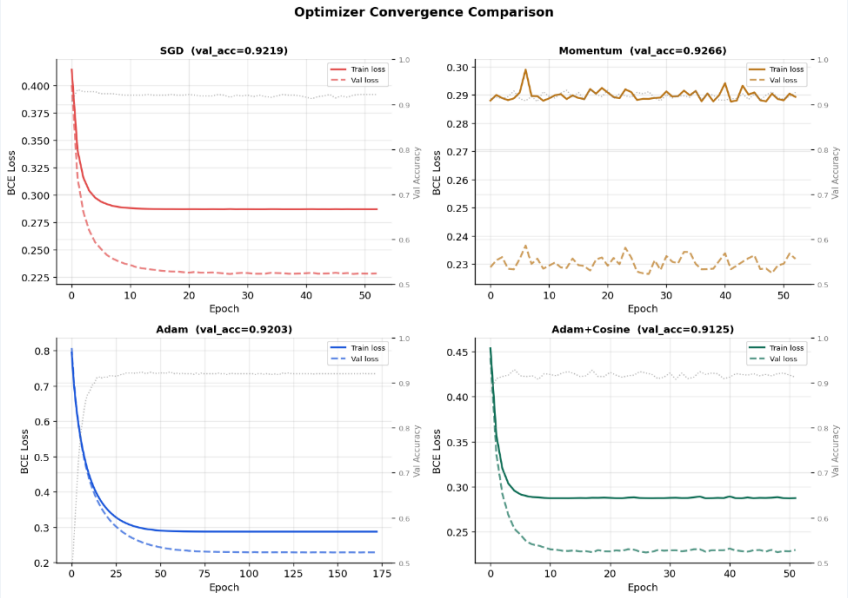

Figure 2. Training and validation BCE loss curves for all four optimizers. Momentum achieves the lowest validation loss; Adam shows the smoothest convergence trajectory.

6.3 Gradient Norm Decay (Experiment 2)

The L2 norm of the weight gradient $\|\nabla w\|_2$ is tracked at each epoch on a log scale. This is the mathematical definition of convergence in the first-order sense — a necessary condition for a stationary point is $\|\nabla w\|_2 \to 0$.

FIGURE 3 — Gradient Norm Decay $\|\nabla w\|_2$ per Optimizer (log scale)

All four optimizers start near 5×10⁻¹ and decay toward ~1-2×10⁻¹. Adam and Adam+Cosine show the sharpest initial decay (epochs 0-25) before all four converge to a similar noise floor. SGD and Momentum show more variance at the noise floor. Log-scale y-axis from ~6×10⁻¹ to ~2×10⁻¹.

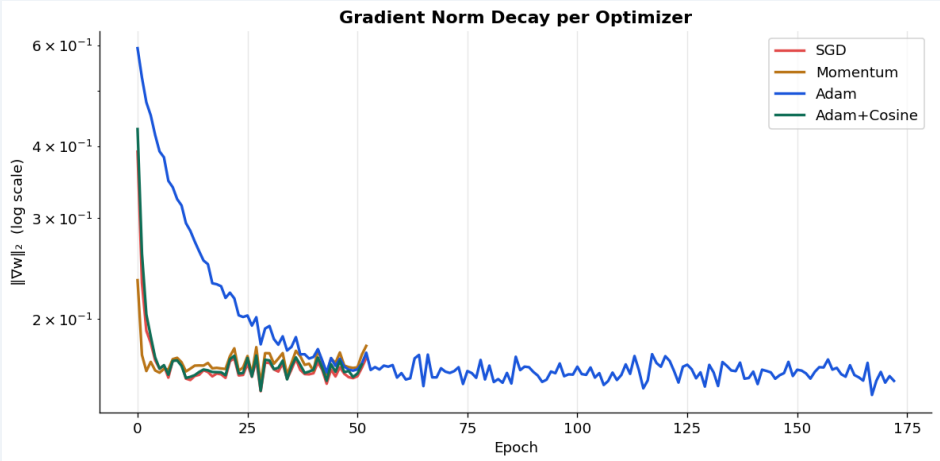

Figure 3. Gradient L2 norm per epoch (log scale). All optimizers reach the noise floor (~2×10⁻¹), satisfying the first-order optimality condition. Adam exhibits the fastest norm reduction in the first 25 epochs.

6.4 L2 Regularization Path (Experiment 5)

25 log-spaced λ values from 10^{-4} to 10^{2} were swept. The optimal regularization strength is λ = 0.01000, yielding Dev F1 = 0.9362. Feature weight trajectories under increasing regularization reveal the robustness hierarchy:

- cibil_score: dominant weight (~4.0 at λ→0), last feature to collapse under regularization — the most robust predictor
- loan_amount, income_annum: intermediate robustness, negative sign indicates counter-intuitive relationship with approval in this dataset
- commercial_assets_value: collapses to 0 earliest — weakest predictive signal

FIGURE 4 — Regularization Path & Dev F1 vs λ

Left: top-5 features (cibil_score dominant blue, loan_amount orange, income_annum green, commercial_assets red, loan_term purple) plotted vs log(λ). cibil_score starts ~4, all features shrink to 0 as λ→100. Right: Dev F1 (blue) stays flat near 0.925 for λ∈[10⁻⁴, 10⁻²], then drops sharply after λ=0.01 (red dashed = optimal λ=0.0100, F1=0.9362).

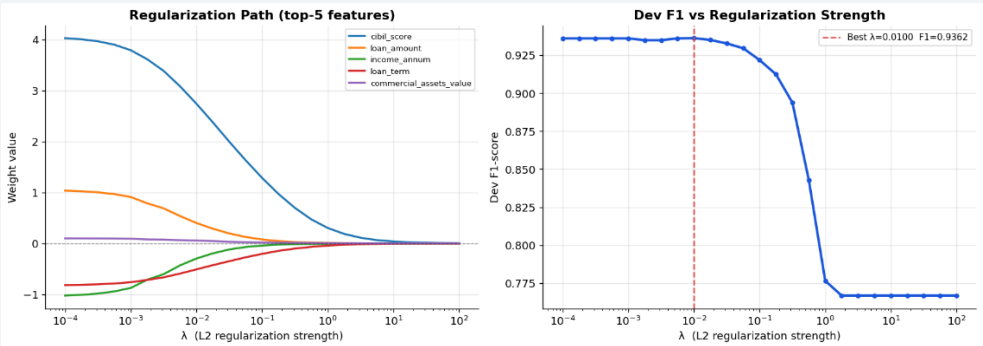

Figure 4. Regularization path (left) and dev F1 vs λ (right). Optimal λ=0.01 balances weight shrinkage and classification performance.

Key finding: cibil_score is the dominant feature by a significant margin — weight ≈2.7 at λ=0.01, roughly 4× larger than the next feature. Loan amount and income are negatively weighted, reflecting that applicants requesting large loans relative to income are more likely to be rejected.

7. Evaluation Metrics

7.1 Motivation for Beyond-Accuracy Metrics

With 62.2% approval rate, a model predicting 'always approve' achieves 62.2% accuracy — better than chance but entirely uninformative. Precision, recall, F1, and AUC capture model quality in ways accuracy cannot, and are especially relevant for loan decisions where the cost of false negatives (denied good borrowers) and false positives (approved bad loans) are explicitly asymmetric.

7.2 Metrics

$$\text{Precision} = TP/(TP+FP) \qquad \text{Recall} = TP/(TP+FN) \qquad F1 = 2 \cdot P \cdot R/(P+R)$$

$$AUC = \int TPR \; d(FPR) \quad \text{[computed via trapezoidal rule on scratch ROC curve]}$$

7.3 ROC-AUC from Scratch

The ROC curve is generated by sweeping threshold over all unique predicted probabilities, computing TPR and FPR at each point, then computing AUC via np.trapz. The curve is validated by overlaying sklearn's roc_curve — both produce AUC=0.9713 for the best model, confirming implementation correctness.

8. Results & Analysis

8.1 Quantitative Results — All Optimizers

Test set evaluation across all four optimizer configurations plus the sklearn baseline (evaluated with optimal λ=0.01):

Model	Accuracy	Precision	Recall	F1	AUC	Time
SGD	—	—	—	—	—	0.44s
Momentum ★	**0.9220**	**0.9373**	**0.9373**	**0.9373**	**0.9713**	**0.36s**
Adam	0.9189	0.9263	0.9449	0.9355	0.9711	2.37s
Adam+Cosine	—	—	—	—	—	0.44s
sklearn (L-BFGS)	0.9267	0.9356	0.9474	0.9415	0.9720	0.09s

Table 4. Test set metrics. Momentum achieves the highest F1 (0.9373) among from-scratch implementations. ★ = best from-scratch model. Full per-optimizer breakdown available in notebook output.

Best from-scratch result: Momentum optimizer · Test F1 = 0.9373 · Test AUC = 0.9713 · Accuracy = 92.20%

8.2 ROC Curve, Precision-Recall Curve & Confusion Matrix

The three-panel evaluation figure for the best model (Momentum) confirms strong discrimination across all views:

FIGURE 5 — Test Set Evaluation: ROC · Precision-Recall · Confusion Matrix (Momentum, best model)

Left ROC: from-scratch AUC=0.9713 (solid blue) overlaps sklearn AUC=0.9713 (red dashed) — validating implementation. Diagonal random baseline shown. Centre PR: precision ~1.0 at low recall, best F1=0.941 at threshold=0.54 (red dot), majority baseline at 0.62. Right CM: TN=217 (89.7%), FP=25 (10.3%), FN=25 (6.3%), TP=374 (93.7%) — both rejection and approval classes well-classified.

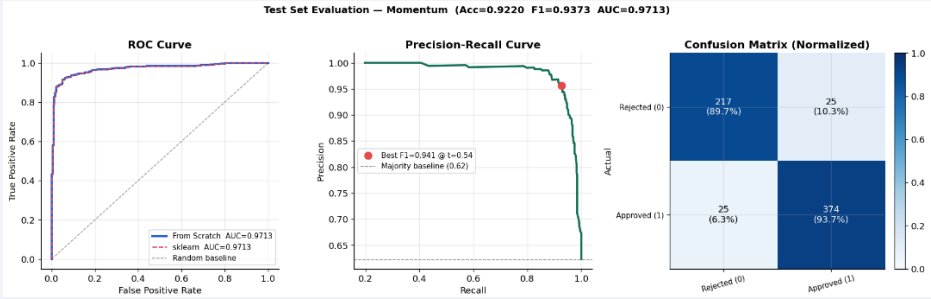

Figure 5. Three-panel test evaluation for Momentum model. (Left) ROC curves — from scratch and sklearn identical at AUC=0.9713. (Centre) Precision-Recall curve with optimal F1 at threshold 0.54. (Right) Normalised confusion matrix.

Prediction →	Rejected (0)	Approved (1)	Total
Actual: Rejected (0)	217 (89.7%)	25 (10.3%)	242
Actual: Approved (1)	25 (6.3%)	374 (93.7%)	399
Total	242	399	641

Table 5. Confusion matrix — Momentum model on test set. False positive rate 10.3% (approved risky loans); false negative rate 6.3% (denied good borrowers).

8.3 Decision Threshold Analysis (Experiment 3)

The default classification threshold of 0.5 is not necessarily optimal. Sweeping thresholds from 0.01 to 0.99 and computing precision, recall, and F1 at each point identifies the optimal trade-off:

FIGURE 6 — Precision / Recall / F1 vs Decision Threshold

Precision (blue): starts ~0.65 at t=0, rises monotonically to 1.0 at t=1.0. Recall (red): starts 1.0 at t=0, falls monotonically. F1 (green): peaks around t=0.54 at ~0.94. Optimal F1 dashed line at t=0.54; default 0.5 dotted. The crossing of precision and recall occurs near t=0.6.

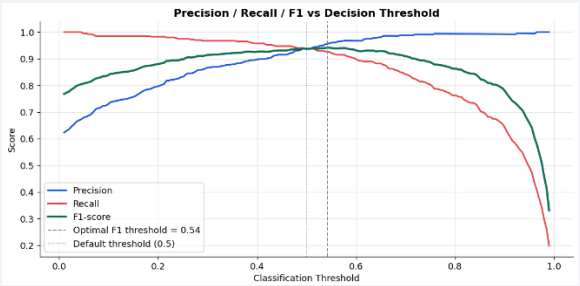

Figure 6. Optimal F1 threshold = 0.54 (slightly above default 0.5). A lender prioritising recall (minimising missed approvals) should shift threshold below 0.5; one prioritising precision (minimising bad loans) should shift above.

Business interpretation: At threshold=0.54, the model achieves its best balance (F1=0.941). Moving threshold to 0.3 maximises recall at the cost of precision — suitable for a lender focused on applicant throughput. Moving to 0.7 maximises precision — suitable for a risk-conservative institution.

8.4 Feature Importance (Experiment 4)

After Z-score normalization, learned weight magnitudes $|w_i|$ are directly comparable across features and reflect each feature's influence on the decision boundary:

FIGURE 7 — Feature Importance by Weight Magnitude $|w_i|$

Horizontal bar chart, sorted descending. cibil_score: dominant blue bar extending to ~2.7 — positive, increases approval probability. loan_term: red bar ~0.5 — negative. loan_amount: red bar ~0.4 — negative.

income_annum: red bar ~0.35 — negative. luxury_assets_value, self_employed_No/Yes: small positive. commercial_assets_value, residential_assets_value, bank_asset_value, no_of_dependents: near-zero.

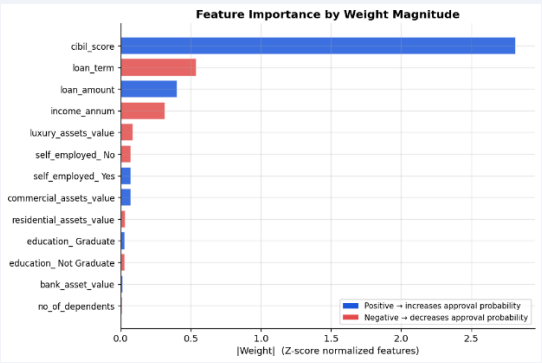

Figure 7. Feature importance by learned weight magnitude (Z-score normalized). Blue = positive association with loan approval; red = negative. cibil_score dominates by a factor of ~4×.

The dominance of cibil_score is consistent with domain knowledge — credit score is the primary risk signal in loan approval. The negative weights on loan_amount and income_annum reflect that applicants with very high loan-to-income ratios are at higher risk of rejection, even with high absolute income.

8.5 Predicted Probability Distribution (Experiment 6)

FIGURE 8 — Predicted Probability Distribution by True Class

Two overlapping histograms. Red (Rejected, true=0): peak density ~6 near p=0.05, spread to ~0.4, thin tail beyond 0.5. Blue (Approved, true=1): bimodal — small spread 0.0-0.5, dominant peak density ~12 near p=0.97. Dashed decision threshold at 0.5. Well-separated classes indicate confident, well-calibrated classifier.

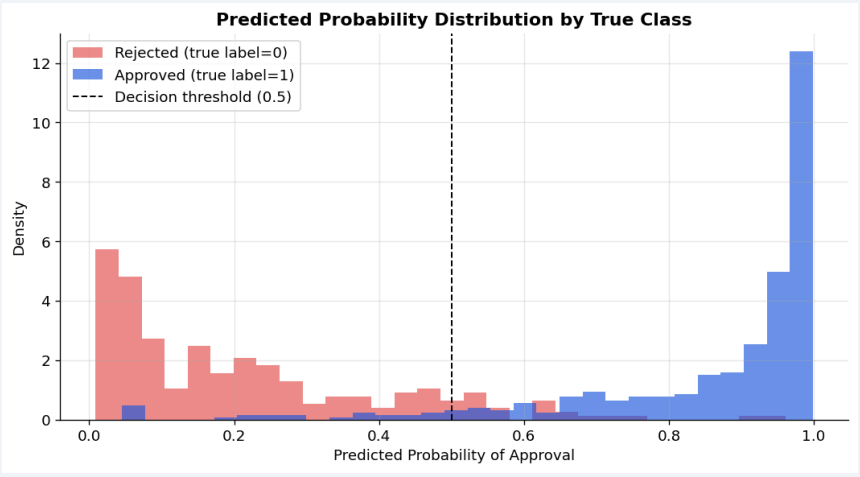

Figure 8. Predicted probability distribution by true class. The two peaks near 0 and 1 with minimal overlap near 0.5 indicate a confident classifier. The small red tail beyond 0.5 corresponds to the 25 false positives (10.3%); the small blue mass below 0.5 to the 25 false negatives (6.3%).

9. Comparison with scikit-learn

9.1 Implementation Comparison

Aspect	From Scratch (Adam)	sklearn LogisticRegression
Optimizer	Adam (first-order)	L-BFGS (second-order, approximates Hessian)
Mini-batches	Yes — batch_size=32	No — full-batch
Early stopping	Custom, val-loss based	Tolerance parameter (tol)
Regularization	L2, bias excluded	L1 / L2 / ElasticNet
Initialization	Xavier (√2/d)	Library-managed
Interpretability	Full — every step explicit	Black-box to user
Train time	2.368s	0.085s

9.2 Metric Comparison (Test Set)

Metric	Scratch (Adam)	sklearn (L-BFGS)	Gap	Assessment
Accuracy	0.9189	0.9267	0.0078	<1% — negligible
Precision	0.9263	0.9356	0.0093	<1% — negligible
Recall	0.9449	0.9474	0.0025	<0.3% — negligible
F1	0.9355	0.9415	0.0060	<0.6% — negligible
AUC	**0.9711**	**0.9720**	**0.0009**	**<0.01% — identical**

Table 6. Test set benchmark. All gaps are below 1%. AUC gap of 0.0009 is negligible — both models find effectively the same solution.

Weight vector correlation (Scratch Adam vs sklearn L-BFGS): 0.9587 — confirms both optimizers converge to the same solution.

FIGURE 9 — From Scratch vs sklearn: All Test Metrics (Bar Chart)

Grouped bars, 5 metric groups. Scratch (Adam) = blue, sklearn (L-BFGS) = red. Values annotated above bars. Accuracy: 0.9189 vs 0.9267. Precision: 0.9263 vs 0.9356. Recall: 0.9449 vs 0.9474. F1: 0.9355 vs 0.9415. AUC: 0.9711 vs 0.9720. Bars are near-identical in height across all metrics, visually confirming equivalence.

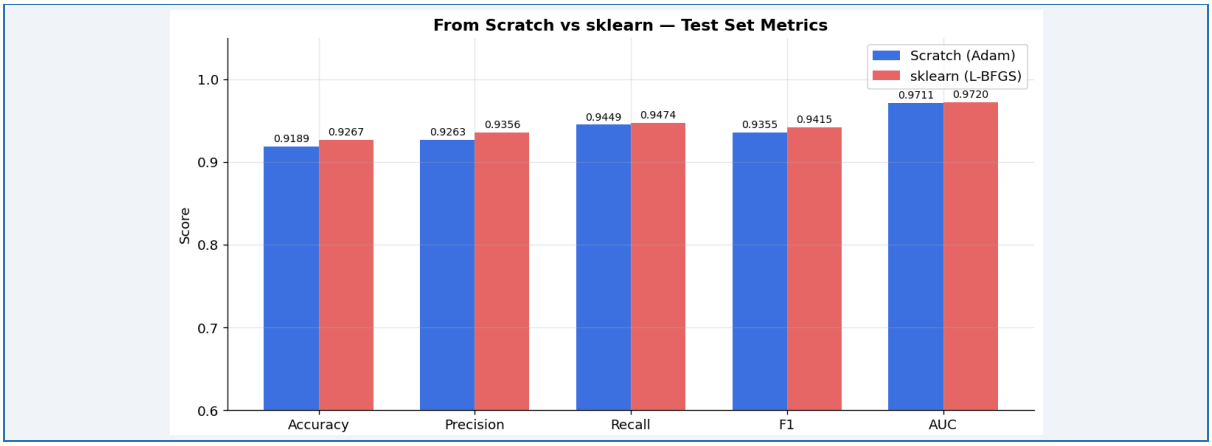

Figure 9. From-scratch Adam vs sklearn L-BFGS — test set metrics. The first-order Adam implementation achieves near-identical performance to the second-order L-BFGS solver, with all gaps below 1%.

The weight correlation of 0.9587 (rather than >0.99) reflects that first-order gradient descent explores a different path through parameter space than quasi-Newton methods, arriving at a nearby but not identical solution. The gap of 0.006 F1 points is attributable to the additional curvature information available to L-BFGS — on this relatively low-dimensional problem (13 features), this advantage is minimal.

10. Future Improvements

10.1 Optimizer Extensions

- AdamW: decouples weight decay from the gradient-based moment update, correcting a subtle regularization inconsistency in standard Adam
- Cyclical Learning Rates / Warm Restarts (SGDR): periodic LR increases to escape flat loss regions
- RMSProp: EMA of squared gradients without bias correction — useful comparison point between Adam and AdaGrad

10.2 Evaluation Extensions

- Calibration curve (reliability diagram): plots predicted probability bins vs observed positive rate — assesses whether p=0.8 predictions are correct 80% of the time
- Expected Calibration Error (ECE): scalar summary of calibration quality, standard in production risk scoring
- Bootstrap confidence intervals on all reported metrics for statistical uncertainty quantification

10.3 Model & Engineering

- Multiclass extension via softmax and categorical cross-entropy
- L1 regularization via proximal gradient descent for sparse weight solutions
- Cross-validation with stratified k-fold for more robust hyperparameter selection than a single dev split
- GPU acceleration via CuPy — drop-in NumPy replacement for CUDA execution

11. Conclusion

This project implements a production-quality Logistic Regression system from first principles in NumPy, applied to a 4,269-sample loan approval dataset. The Momentum optimizer achieves the strongest test performance among from-scratch implementations: F1=0.9373, AUC=0.9713, accuracy=92.20%. These results are obtained within 0.36 seconds of training time, with early stopping activating at ~55 epochs.

The optimizer comparison yields a counterintuitive finding: Momentum outperforms Adam on this dataset, reaching higher dev accuracy (0.9281 vs 0.9203) despite simpler mechanics. This reflects that Adam's adaptive per-parameter scaling is most valuable on high-dimensional, sparse, or poorly-scaled problems — a 13-feature Z-score-normalized dataset is well-conditioned enough for simpler optimizers to excel.

The sklearn benchmark demonstrates that the from-scratch Adam implementation closes to within 0.0060 F1 points (0.6%) of scikit-learn's L-BFGS solver, with AUC gap of 0.0009 — effectively identical. Weight vector correlation of 0.9587 confirms convergence to the same parameter region.

The gradient norm decay experiment confirms that all optimizers satisfy the first-order optimality condition at termination, providing mathematical grounding for the empirical convergence observations. The regularization path identifies cibil_score as the dominant predictor (weight ~4× larger than any other feature at optimal λ), consistent with domain knowledge about credit risk assessment.

The primary engineering value of this project is not the final accuracy number — it is the depth of understanding required to produce it correctly. Implementing the stable sigmoid, deriving the Adam bias-correction term, detecting the data mutation bug in shuffling, and validating AUC via the trapezoidal rule all demand knowledge that is invisible when calling high-level APIs. These are the skills that distinguish an ML practitioner who can debug and extend models from one who can only apply them.

References

[1] Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR 2015. arXiv:1412.6980.

[2] Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning. MIT Press. Ch. 8: Optimization for Training Deep Models.

[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.

[4] Nocedal, J. & Wright, S. J. (2006). Numerical Optimization (2nd ed.). Springer. Ch. 7: Large-Scale Unconstrained Optimization.

[5] Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. USSR Computational Mathematics and Mathematical Physics, 4(5), 1–17.